

## Assignment 4

Imagine you're working for the Pennsylvania Department of Transportation (PennDOT) and they're building a model to simulate car traffic on the roads.

The model is a C++ program that was started by a previous employee a couple years ago. Work stalled because the program became too slow to run and the previous employee gave up on it! You have been given this C++ program and **your job now is to speed up the program** without changing anything else about how it works.

### Example Run

Running the simulation for just 1 week (604800 seconds) takes almost 15 minutes!

```
$ time ./traffic-simulation 604800
--- Traffic Simulator ---
Computing crash probabilities from database...
Day: Friday, probability of a crash: 0.094086
Day: Wednesday, probability of a crash: 0.0851254
Day: Thursday, probability of a crash: 0.0862007
Day: Tuesday, probability of a crash: 0.0917563
Day: Monday, probability of a crash: 0.0827957
Day: Saturday, probability of a crash: 0.0887097
Day: Sunday, probability of a crash: 0.0645161
Simulating traffic...
99%
Computing sample statistics...
99%
---Simulation Finished---
    duration: 168hrs 0mins 0secs
    road capacity: 200
    avg new cars / second: 4
    number of crashes: 0
    total number of cars simulated: 2421386
    The number of moments (individual seconds) in which there were 0 cars: 0
    The number of moments (individual seconds) in which there was exactly 1 car: 11
    The number of moments (individual seconds) in which there were only 2 cars: 71
    The number of moments (individual seconds) in which there were 10 cars or
fewer: 90746 (about 15% of the entire simulation time.)
HTML file generated successfully: plot.html

real    14m7.087s
user    14m3.766s
sys     0m3.277s
```

The program computes the correct data, the simulation is accurate. But it takes waaay too long to be useful. The PennDOT higher-ups would like to simulate an entire year in a minute or two. You need to get the program to compute the exact same values in much less time. The output for a given input (like 604800) should remain unchanged.

**Pretend you received the following notes / details from your boss**

- The program reads from a database `CRASH_LANCASTER_2024.db`
  - A couple years back an intern at the company wrote a simple python script that converts the database from sqlite (.db) to comma-separated-values (csv). You can open the csv file with any spreadsheet program, to see the contents of the database. But, the intern lost the python conversion script.
  - The C++ simulation should work correctly even with a different database (perhaps from a different year) substituted in place. Of course the results of the simulation might differ in that case.
- Your boss said, “the database part seems like a drunk monkey wrote it.”
- To compute the crash probabilities for a given day, count the number of crashes that occurred on that day where the automobile count was not zero. Divide this number by the total number of incidents in the database. It seems like whatever part of the code does this does it correctly.
- Poisson, and Binomial distributions are used as the backbone of the simulation. You don’t need to fully understand these details and you shouldn’t change what the simulation uses them for. In case you find it useful, here is some more information.
  - The simulation is concerned mostly with how many cars are on the road at a given time (second), and how many crashes occur.
  - Each second, there is a chance that some new cars enter the road. The number of cars that enter is given by selecting a value from the Poisson distribution ([https://en.wikipedia.org/wiki/Poisson\\_distribution](https://en.wikipedia.org/wiki/Poisson_distribution)) with  $\lambda = 4$ .
  - Each second, a certain number of cars exit the road. The number of cars that exit is given by selecting a value from the binomial distribution ([https://en.wikipedia.org/wiki/Binomial\\_distribution](https://en.wikipedia.org/wiki/Binomial_distribution)) with  $n =$  the number of cars, and  $p = 0.298 * (1 - n/200)$ . 200 is the capacity of the road. So when the number of cars approaches 200 the probability of each car leaving goes down. This simulates traffic-jam like conditions. If  $n = 200$  then the road is at capacity, there is complete traffic stoppage, and no cars can exit at all.
  - Each day, there is a chance of a car accident. The probability of a car accident is determined for each day of the week based on the history of car crashes in the database. For example, the chance of a crash on a Monday is 0.082... (8%). A random number is generated between 0 and 1. If it is below 0.08 then a crash occurs. When there is a crash no cars can exit the road for 10 minutes.
- The simulation is not perfect. A lot of aspects of realistic traffic simulation are missing. But before working on that it’s more important to get the program running faster.
- If there are any recursive functions maybe get rid of them. Recursion is usually slow and I just don’t like it very much.
- Of course the program will run faster on a more powerful computer. For this project make sure you’re measuring the improved running time on the same computer.

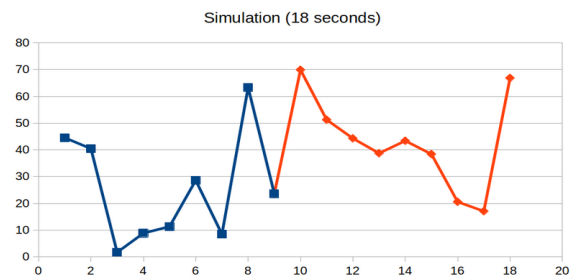
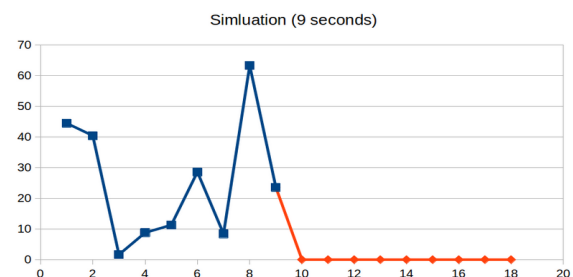
- Use the time command (like the example runs show) to measure how fast the code is and to keep track of your progress as you speed it up
- The output in the terminal should be a good clue as to the program running correctly. Whatever code you change, the numbers printed on the terminal should remain the same for a given duration.
- Optionally, you can view the HTML web-page (plot.html) that the program generates to be very sure you aren't changing any computed values in the simulation.
- The program comes with some tests to further ensure you haven't made a problematic change to the codebase. You can at, any point, use the Makefile to compile the tests (`Make tests`) and run them (`./tests`). Obviously, they should all pass.
- There are a lot of files, the `main()` method and the core logic of the simulation is in `Sim.cpp`. That's a good starting point.

### Deterministic Program

This program is deterministic. If you give the same inputs, it will give the same outputs. The inputs are the duration simulation duration, which is a number of seconds given on the command line, and the crash database, "CRASH\_LANCASTER\_2024.db" which is hard-coded into the program.

The simulation carries out the same exact way every time for a given set of inputs. Even though "random" numbers are used in the simulation, a proper random seed is not set so the same "random" numbers will be generated each time the program is run.

For example, the figures on the left demonstrate a simulation that ran for 9 seconds vs one that ran for 18 seconds. Notice, the blue segment is identical for both.



Although a real traffic simulation would use truly random numbers, this program is kept deterministic in order to make the task of speeding it up easier. You can be sure that you aren't changing the code inappropriately if the program is still generating the same output for the same input.

## **Example Run #2**

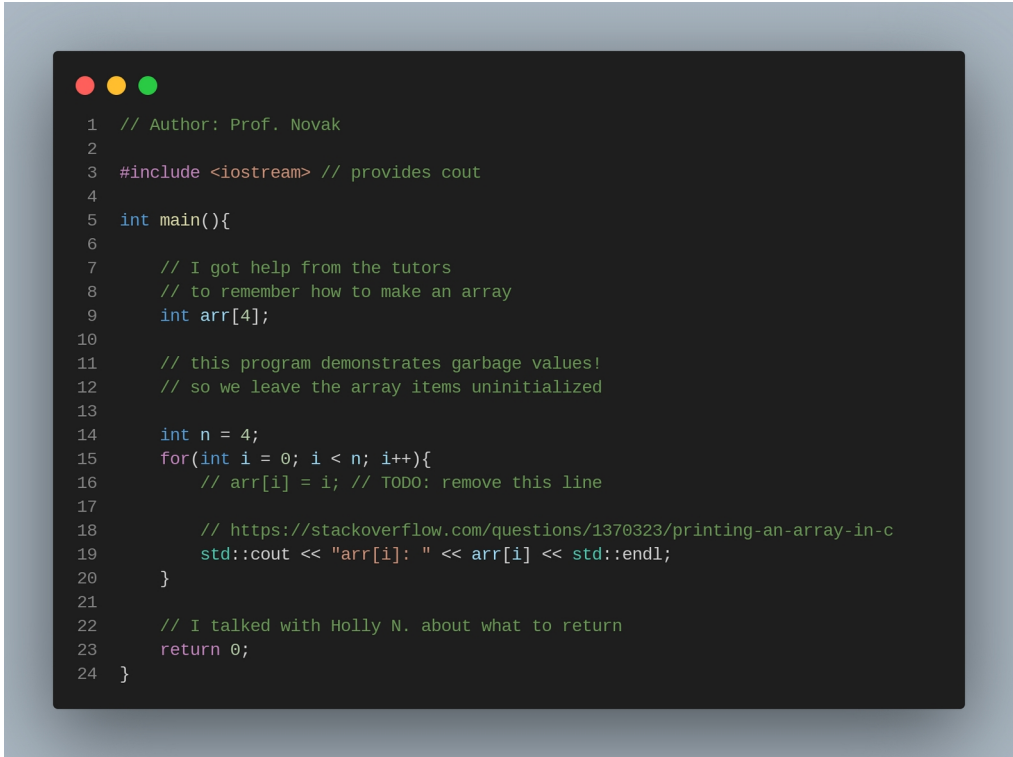
Running the simulation for 1 day (86400 seconds)

```
$ time ./traffic-simulation 86400
--- Traffic Simulator ---
Computing crash probabilities from database...
Day: Friday, probability of a crash: 0.094086
Day: Wednesday, probability of a crash: 0.0851254
Day: Thursday, probability of a crash: 0.0862007
Day: Tuesday, probability of a crash: 0.0917563
Day: Monday, probability of a crash: 0.0827957
Day: Saturday, probability of a crash: 0.0887097
Day: Sunday, probability of a crash: 0.0645161
Simulating traffic...
99%
Computing sample statistics...
99%
---Simulation Finished---
    duration: 24hrs 0mins 0secs
    road capacity: 200
    avg new cars / second: 4
    number of crashes: 0
    total number of cars simulated: 345709
    The number of moments (individual seconds) in which there were 0 cars: 0
    The number of moments (individual seconds) in which there was exactly 1 car: 3
    The number of moments (individual seconds) in which there were only 2 cars: 10
    The number of moments (individual seconds) in which there were 10 cars or
fewer: 12939 (about 14% of the entire simulation time.)
HTML file generated successfully: plot.html

real    0m36.878s
user    0m35.753s
sys      0m1.124s
```

## Proper Comments and Citations

Your code should have comments that explain “why” the code is written the way it is. You should also use comments to cite sources you used such as websites, tutors, and classmates. You can focus your comments on the areas of the code which you changed.



```
1 // Author: Prof. Novak
2
3 #include <iostream> // provides cout
4
5 int main(){
6
7     // I got help from the tutors
8     // to remember how to make an array
9     int arr[4];
10
11     // this program demonstrates garbage values!
12     // so we leave the array items uninitialized
13
14     int n = 4;
15     for(int i = 0; i < n; i++){
16         // arr[i] = i; // TODO: remove this line
17
18         // https://stackoverflow.com/questions/1370323/printing-an-array-in-c
19         std::cout << "arr[i]: " << arr[i] << std::endl;
20     }
21
22     // I talked with Holly N. about what to return
23     return 0;
24 }
```

**Submission:** Your program will be submitted using GitHub classroom and canvas.

After following the instructions on canvas you should have a repository cloned to your computer which contains these assignment instructions. That same repository will be used for you to submit your solutions.

1) Edit and write code in the repo on your computer. Use `git add`, `git commit`, and `git push` to put that code onto github. I have access to the repo. You can make as many commits as you like for this assignment with no penalty.

I will grade the final commit made before the due-date.

2) You should navigate to the assignment on canvas and upload your video recording.